

Introdução Funções: Aula 04

COMANDO.C

1 Comparando a main() e o printf()

Observe a instrução a seguir:

```
printf("Olá, Mundo!");
```

O `printf()` é uma das primeiras linhas de código que você vai digitar enquanto aprende qualquer linguagem de programação, com a célebre frase `Olá, Mundo!`. Mas se formos analisar com atenção, veremos que o `printf()` tem algumas similaridades e diferenças com a `main()`.

COMPARATIVO	
<code>MAIN(){ }</code>	<code>PRINTF();</code>
POSSUI ()	POSSUI ()
POSSUI {}	NÃO POSSUI {}
NÃO POSSUI ;	POSSUI ;

Comparando ambas as instruções, notamos que elas têm em comum **um nome**, destacado em azul, e os **parênteses**, isso acontece pois as instruções `main()` e `printf()` são funções em C.

As funções podem ser **Definidas**, **Chamadas** e **Declaradas** em nossos códigos.

2 Definindo uma função

Para definir uma função, precisamos entender sua **estrutura**. Como vimos anteriormente, precisamos de um nome e os sinais de parênteses, que podem ou não ter parâmetros dentro.

```
tipo NomeDaFuncao(tipo parametro1, tipo parametro2){  
    // . . .  
    return ValorDeRetorno;  
}
```

No caso acima, temos uma função chamada `NomeDaFuncao()`, que recebe dois argumentos, pois tem `parametro1` e `parametro2`. Ademais, ela retorna o valor da variável `ValorDeRetorno` para quem a chamou.

Argumentos e parâmetros possuem uma diferença entre si, embora sejam comumente usados como sinônimos. Os parâmetros estão presentes na declaração de funções e servem para definir o tipo de variáveis que ela vai aceitar, enquanto os argumentos são as próprias variáveis que passamos para as funções, o conteúdo em si.

```
int main(){  
    NomeDaFuncao(5); // ← 5 é o dado de fato fornecido, o argumento  
    // ...  
}  
  
int NomeDaFuncao(int x){ // ← variavel x vai receber o argumento  
    // Código da função  
}
```

Definir uma função é necessário para dizer ao compilador quais instruções ela deve executar quando chamada, isto é, escrever o corpo da função (sequência de declarações e instruções).

No caso da `main()` nós apenas definimos ela no nosso código, pois, como é o *entry point* (ponto de entrada) do programa, a chamada da `main()` ocorre por parte do sistema operacional.

3 Chamando uma função

Agora que escrevemos o que a nossa função deve executar, como fazemos para usá-la? Para isso, precisamos realizar uma chamada de função.

A maneira como chamamos a função `printf()` é equivalente a chamada de qualquer outra função, portanto, vamos modificar o nosso pseudocódigo anterior para:

```

int main(){
    // . . .

    NomeDaFuncao(arg1, arg2); // ← Chamando nossa função

    return 0;
}

// Abaixo, definimos nossa função
tipo NomeDaFuncao(tipo parametro1, tipo parametro2){
    // . . .
}

```

Como podemos ver, além de chamar nossa função usando `NomeDaFuncao()`, também fizemos a passagem de argumentos, de supostas variáveis chamadas de `arg1` e `arg2`, pois o nosso pseudocódigo anterior previa essa implementação.

Chamar uma função é escrevê-la no código para dizer ao compilador que naquele momento ela deve ser executada. Funções definidas em um programa, mas que nunca são chamadas, não passariam de linhas extras de códigos. Ao chamar uma função, nós podemos ou não lhe passar argumentos, a critério da implementação.

Ainda não entraremos em detalhes sobre o comportamento e implementações para lidarmos com os valores de retorno de uma função.

4 Declarando uma função

A declaração de funções nada mais é do que o “cabeçalho” de uma função, nela definimos seu nome, tipo e sua lista de parâmetros, mas por que isso é necessário?

```

int main(){
    soma(1,2);
}

int soma(int n1, int n2){
    return n1 + n2;
}

```

Vamos compilar o código acima, teremos como saída:

```

definicao.c: In function 'main':
definicao.c:2:5: error: implicit declaration of function 'soma'
[-Wimplicit-function-declaration]
  2 |     soma(1,2);
    |     ^~~~

```

Até agora fizemos tudo corretamente, mas por que o código não compila?

Pois bem, a saída do GCC diz que há uma declaração **implícita** de uma função, mas ele não achou em nenhum lugar do nosso código onde ela estaria **explicitamente** declarada. Nós não conseguimos usar uma função sem ter avisado previamente ao compilador que ela existe.

Bom, vamos então escrever um “rascunho” ou uma espécie de “cabeçalho” para essa nossa função de soma, para que o compilador esteja ciente da sua existência. A forma para definir uma função segue:

```

tipo NomeDaFuncao(tipo parâmetro);

```

Vamos repetir isso para a nossa função soma():

```

int soma(int n1, int n2);

```

Note, uma função pode ter quantos parâmetros forem necessários para que se cumpra seu propósito, como queremos somar dois números, usamos n1 e n2 para carregar essas informações.

Agora, implementamos no código:

```

int main(){
    soma(1,2);
}

int soma(int n1, int n2){
    return n1 + n2;
}

int soma(int n1, int n2); // ← Estamos definindo ela aqui.

```

Compilamos e teremos:

```

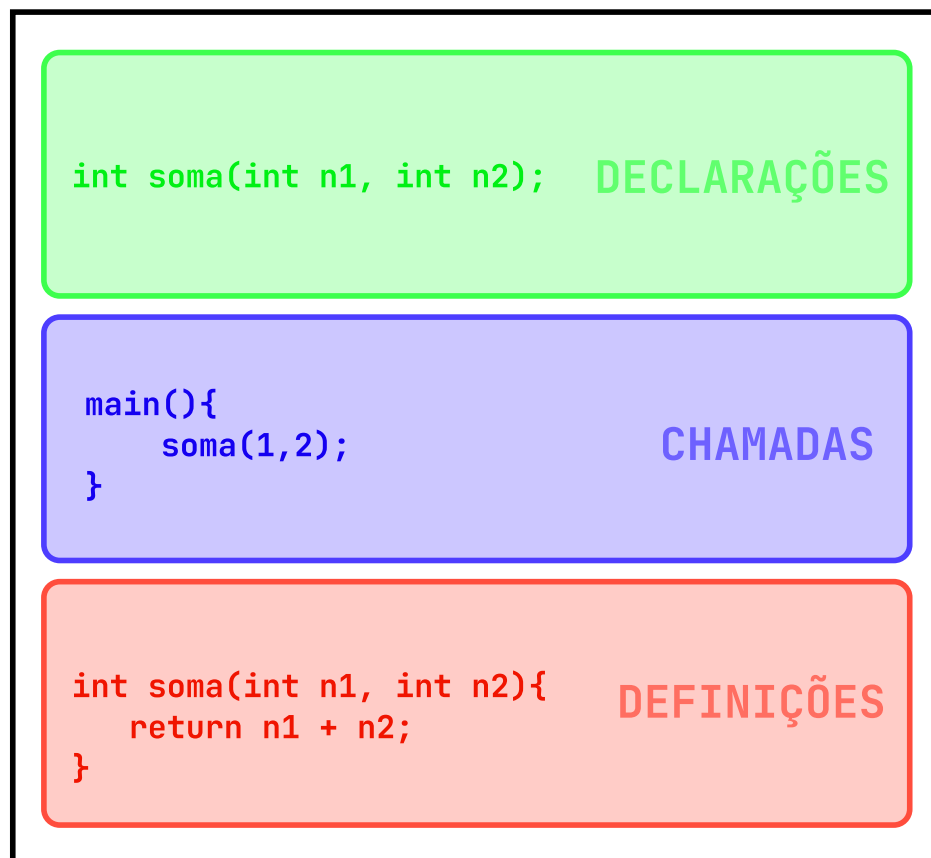
definicao.c: In function 'main':
definicao.c:2:5: error: implicit declaration of function 'soma'
[-Wimplicit-function-declaration]
  2 |     soma(1,2);
    |     ^~~~

```

Novamente o mesmo erro, o que está acontecendo?

Acabamos de dizer que definir uma função é como escrever um “cabeçalho” para ela, assim o compilador tem ciência dessa implementação e fará o que for necessário para que tudo funcione corretamente. Seguindo essa linha de raciocínio, como nós poderíamos avisar “Ei, compilador! Existe uma função soma() no meu código”, sendo que só fizemos isso **depois** que chamamos ela na main()? Ou seja, existe uma ordem para **declarar, chamar e definir** uma função:

main.c



Dessa maneira, mantemos nosso código organizado e de fácil entendimento para outros programadores, além de permitir que o compilador faça seu trabalho da maneira mais apropriada possível.

5 Retornos da `main()`

Como vimos na aula, a estrutura básica da função `main()` é essa, mas preste atenção no valor de retorno que estamos fazendo:

```
int main(){
    printf("Hello, World!");
    return 0; //← retornando 0
}
```

Historicamente falando, para que o *exit status* (status de saída) da `main()` seja interpretada pelo sistema como “sucesso”, o valor a ser passado é `0`, no caso do valor de retorno ser diferente de `0`, o status é de “falha”. Por convenção, os números `1` e `-1` são usados para isso, mas fica a critério da implementação o que deve ser interpretado.

Os valores de retorno são tremendamente úteis para nós programadores, pois assim conseguimos controlar melhor as interações do usuário e o fluxo do programa, se antevendo para erros que prejudicam a integridade da aplicação.

Para além da estrutura mostrada acima, devemos adicionar:

- o tipo dela, no caso `int` é o recomendado;
- os parênteses, no nosso caso sem nada entre eles;
- e por fim um valor de retorno.

Portanto, o menor programa C possível é:

```
int main(){
    return 0;
}
```

Uma curiosidade, você sabia que existem compiladores que aceitam esses tipos de chamadas da função `main()`?

```
void main(){
}
```

```
main(){
}
```

Este tipo de implementação não segue os padrões de conformidade e ainda pode ocasionar erros nas suas aplicações. A ISO da linguagem C trata esses tipos de implementações como incorretas.

Com isso, cobrimos os pontos iniciais para começarmos a entender e trabalhar com funções, mas ainda nos faltam conhecimentos importantes, como tipagem de dados e passagens de controle. Porém não se preocupe, esses serão assuntos para nossos próximos vídeos e PDFs.

Mas espere, ainda não acabamos, acompanhe nosso próximo PDF, onde falamos sobre *flags* e bibliotecas em C.